

Deterministic Noise Cycles Make Golden-Ticket Search Work on Long-Horizon Manipulation

Yuan Yang Ma (a.k.a. Eddy Ma)
Wiltechs Technology
eddy.ma@wiltechs.com

Abstract

Chunked flow-matching policies redraw their initial noise x_1 at every replan. We show that this redraw carries a large share of a policy’s success rate on long-horizon tasks, and that the golden-ticket intervention – replacing the redraw with one searched constant vector – therefore fails to transfer there. On LIBERO-Long, a random frozen vector passes 6–12.5% of canonical initial states where the per-chunk redraw passes 90%, and one such vector takes a task from 55% to 15%. We attribute the collapse to *repetition* rather than to the choice of vector: training never presents two consecutive chunks with identical noise, and a long episode shares one draw across roughly 76 chunks. Cycling m fixed vectors, with chunk k using element $k \bmod m$, restores the regime while keeping the rollout fully deterministic. Cycling alone is not an improvement – an unsearched 4-cycle scores 63.5% against the redraw’s 71.0% over ten tasks – but it raises a candidate’s per-layout pass rate from 6% to 61%, which is what makes search feasible at all. On a frozen policy built on a 500M-parameter vision-language encoder, searched 4-cycles take LIBERO-Long from 71.0% to 79.0% with all nine searched tickets active, winning 32 discordant initial states to 16 (two-sided sign test, $p = 0.029$), and to 81.0% when the one ticket that loses to the redraw falls back to it ($p = 0.0022$) – with no weight updates and bit-reproducible rollouts. Across all four LIBERO suites the policy rises from 83.5% to 92.3%. We further report that a search rate is a maximum over candidates and is systematically optimistic: three tickets the search ranked as winners did not beat the unmodified policy on states outside the search, and one cost 20 points.

1 Introduction

A flow-matching policy turns a sample $x_1 \sim \mathcal{N}(0, I)$ into an action chunk by integrating a learned velocity field. The draw is usually treated as plumbing. Patil et al. [12] showed it is not: for a pre-trained SmolVLA policy [16] on LIBERO [10], searching over x_1 and freezing the single best vector per task lifts average success from 84.7% to 96.2% with the weights untouched. The intervention is cheap, requires no gradients, and makes the rollout deterministic.

It also, as published, does not work on long-horizon tasks. Those were not in the evaluation: Patil et al. report Object, Goal and Spatial and omit LIBERO-Long. When we applied the same intervention to LIBERO-Long on our own policy, success collapsed. A random frozen vector solved 3 of 20 canonical initial states on a task where the unmodified policy solved 11, and a tiered search over 64 candidates found only 8 that cleared a single layout – a 12.5% per-layout pass rate against the redraw’s 90%. At that rate the search has nothing to select from, so the intervention has no long-horizon version at all.

This paper identifies the cause and removes it. The cause is not that long tasks have no good vector; it is that one vector is reused too many times. A chunked policy replans every `n_action_steps` environment steps, so a successful LIBERO-Long episode draws about 76 independent x_1 vectors, against 33 on Spatial and 31 on Goal. Freezing x_1 replaces all 76 with one, and the model has never, in training, conditioned on two adjacent chunks that share their noise.

The fix follows directly. Hold m fixed vectors and let chunk k use element $k \bmod m$. Nothing is sampled at inference, so the rollout stays bit-reproducible, but consecutive chunks no longer share a draw. At $m = 4$ the per-layout pass rate over a pool of candidates rises from 6% to 61%, and search becomes feasible.

The honest version of the result has two halves, and the second is what the contribution rests on. Cycling by itself does *not* improve the policy: a single unsearched 4-tuple applied to all ten LIBERO-Long tasks scores

63.5% against the redraw’s 71.0%, losing 41 discordant layouts to 26. Cycling only buys a searchable regime. The gain comes from searching within it: per-task searched 4-cycles reach 79.0%, winning 32 discordant layouts to 16 ($p = 0.029$) with every searched ticket active. One of the nine costs points; letting that task fall back to the redraw gives 81.0% and 30 to 10 ($p = 0.0022$).

Where this sits. We are not claiming a better way to train a VLA. The policy here is a small, frozen, open-weights model, and every number we report was obtained without touching its parameters. What we are claiming is that the inference-time noise of a chunked flow-matching policy is a usable, measurable axis on long-horizon tasks, that it is usable only once the repetition constraint is removed, and that reporting it correctly requires held-out initial states – because a search rate is a maximum over candidates and overstates a ticket by construction. The last point is not a caveat we add at the end; it changed three of our own conclusions.

Contributions.

1. We show that freezing x_1 collapses long-horizon success, and that the collapse is caused by repetition across chunks, not by the choice of vector (§4.3).
2. We introduce deterministic m -cycling, which restores the per-chunk regime while keeping rollouts bit-reproducible, and we measure that it raises the candidate pass rate from 6% to 61% without, on its own, raising success (§4.4).
3. We extend golden-ticket search to m -tuples with exact pruning and stratified tiers, and report the first long-horizon result for the intervention: 71.0% $\rightarrow$ 79.0% on LIBERO-Long, frozen weights (§4.5).
4. We document selection inflation in this setting: three tickets the search ranked as winners did not beat the unmodified policy on held-out states, one by 20 points, and a two-batch paired comparison against the redraw catches them (§4.7).

A smaller result supports these. Before any noise intervention, the 8-expert decoder configuration we use outperforms the earlier 4-expert configuration by 7.7 points across the four suites (§4.2), so the baseline the tickets improve on is the strongest of its lineage, not a weak one.

2 Related Work

Acting on the noise, not the weights. Mao et al. [11] observed that some initial latents in a diffusion model are systematically better than others, carrying semantic structure before any denoising – a lottery-ticket phenomenon [5] in the noise rather than in the parameters. Patil et al. [12] brought this to robot policies: a per-task constant x_1 , found by random search, improves a frozen flow-matching VLA. A parallel line treats the noise as a continuous control variable and optimises it with reinforcement learning [15, 18]; those methods act per step and are not deterministic at inference. Our work sits with the first line – a frozen, searched artefact – and extends it along the axis the first line did not cover.

The chunking structure. Diffusion and flow-matching policies emit action chunks rather than single actions [3, 6, 9, 17], and re-run the generative model every `n_action_steps` environment steps. The number of draws per episode is therefore a property of the task’s length, not of the model. To our knowledge, no prior work treats the *sequence* of per-chunk draws as the object to design; this paper does.

Small VLAs. SmolVLA [16] showed that a sub-billion-parameter vision-language backbone is enough for competitive LIBERO performance, in contrast to larger generalist policies [1, 14]. Our policy follows that premise – a frozen 500M encoder [7] with a mixture-of-experts flow-matching head – which is also what makes an inference-time search affordable: every candidate evaluation is a rollout, and the method’s cost scales with the cost of running the policy.

Search. Our search is random search with sequential halving [8] over tiers of initial states, with exact arithmetic pruning and Clopper–Pearson bounds [4] as an optional, more conservative prune. Patil et al. [12] compare random search against CEM and use random search for their LIBERO results; we do the same and do not revisit the choice.

3 Method

3.1 Policy

We evaluate **WILRO-MoE**, a flow-matching VLA with the following structure. A frozen SmolVLM2-500M-Video-Instruct model [7] reads both camera images and the instruction once per observation and exposes the keys and values of all 32 of its language-model layers. The action head is a dense mixture of 8 diffusion-transformer experts of 4 layers each; expert e cross-attends into VLM layers $4e$ to $4e+3$, reading the cached keys and values directly, and a router weights the experts’ velocity predictions. A trainable ResNet-18 adds agent- and wrist-camera tokens to the decoder’s sequence, alongside the current and previous robot state. Of the policy’s 1.11B parameters, 460M (the VLM) are frozen and 649M are trained (Figure 1). The head is trained with the conditional flow-matching objective [9]

$$x_t = t x_1 + (1 - t) a, \quad \mathcal{L} = \mathbb{E} \|v_\theta(x_t, t, c) - (x_1 - a)\|^2,$$

where a is the ground-truth action chunk and c the observation and language conditioning. At inference the policy integrates the learned field from x_1 back to $t = 0$ in 10 Euler steps, producing a chunk of $H = 64$ actions of dimension $D = 7$, of which the first `n_action_steps` = 2 are executed before replanning. Full architecture details are in Appendix A. Every experiment in this paper uses one public checkpoint with weights frozen throughout.

3.2 Golden tickets and their failure mode

The golden-ticket intervention replaces the draw $x_1 \sim \mathcal{N}(0, I)$ with a fixed $t^* \in \mathbb{R}^{H \times D}$, identical at every chunk of every episode of a task. The vector is found by search and then frozen.

Let N be the number of chunks in an episode. The unmodified policy draws N independent vectors; the intervention supplies one vector N times. The two are indistinguishable when $N = 1$ and diverge as N grows – and N is set by the task. Measured over successful episodes on our checkpoint, the mean is 32.7 chunks on Spatial, 31.4 on Goal, 41.7 on Object, and 75.9 on LIBERO-Long. The training distribution contains no example in which consecutive chunks share a noise vector, because each training sample draws its own.

3.3 Deterministic cycling

We generalise a ticket from a vector to an ordered tuple $T = (t_0, \dots, t_{m-1})$, $t_i \in \mathbb{R}^{H \times D}$, and set the k -th chunk’s initial noise to

$$x_1^{(k)} = t_{k \bmod m}.$$

The counter k resets at episode start, so a rollout’s noise sequence depends only on T – not on a global seed, not on the previous episode’s length, and not on hardware. The case $m = 1$ recovers the original intervention exactly; we verified that the $m = 1$ code path and a plain single ticket built from the same seed produce bit-identical per-episode outcome vectors. The case $m \rightarrow \infty$ approaches the per-chunk redraw. Nothing is sampled at inference for any finite m , so determinism is preserved throughout.

3.4 Search

Candidates are drawn as $T \sim \mathcal{N}(0, I)^{m \times H \times D}$, seeded per task so that two machines searching the same task draw the same pool and two machines searching different tasks do not. Evaluation proceeds in tiers of initial states of increasing size; a candidate advances only if its cumulative success rate clears a floor derived from the unmodified policy’s measured rate on exactly the layouts that candidate has run.

Two prunes are available. The *point* prune drops a candidate when $w/n < f$ for wins w , runs n and floor f . The *bound* prune drops it when the Clopper–Pearson upper bound at a chosen confidence falls below f , which is strictly more conservative and keeps more candidates alive. In addition an *exact* prune drops any candidate for which $(w + M - r)/n_{\text{end}} < f$, where M is the tier’s total episode budget and r the episodes already run – these candidates are arithmetically dead and evaluating them further cannot change the outcome.

3.5 Reporting protocol

LIBERO defines 50 canonical initial states per task. We partition them once and keep the partition fixed for every experiment in this paper:

States	Role	Used for
0–19	reported	every number in §4; never used by the search tiers
20–34	search	the tiers of §3.4 and the baselines its floors use
35–49	certification	the independent second batch of §4.7

The search tiers never touch a reported state. After a search, the ticket for a task may be adjusted using states 0–19 – choosing among the top candidates of its pool, or falling back to the redraw where the ticket loses to it; Limitation 4 discusses what this implies. Comparisons between the redraw and a ticket are paired by initial state and tested with a two-sided sign test (McNemar) on the discordant pairs. We report this rather than a difference of proportions because the two conditions share the layout, and the paired test is the one with power at $n = 20$.

4 Experiments

4.1 Setup

All results use the checkpoint ISdept/wilro-wilromoe-8x4-22k-obs2, weights frozen, $H=64$, $D=7$, 10 inference steps, `n_action_steps=2`, LIBERO [10] through the LeRobot harness [2] at `control_freq=10` Hz, matching the dataset, the environment’s own step cap, and `fixed_init_states` with seed 10000 so that a layout index names the same scene in every run. Every reported evaluation ran on an NVIDIA L4 GPU; ticket searches also ran on other GPUs. The software stack is pinned: PyTorch 2.7.1, LeRobot 0.4.0, robosuite 1.4.1, MuJoCo 3.1.6, LIBERO 0.1.0, Transformers 5.18.0, Gymnasium 1.3.0, CUDA 12.6. Input images are rendered at 384×384 and 10 environments are stepped in parallel.

The unmodified policy scores **83.5%** averaged over the four suites on states 0–19 (Table 4, left column). Every ticket result in this paper is measured against that policy on those states.

A note on `n_action_steps`. The checkpoint ships `n_action_steps=64`, under which the policy replans a handful of times per episode and scores near zero. Three results in this project were lost to that default before our evaluation script learned to warn. Any reproduction must set it explicitly.

4.2 The base policy

Before any noise intervention, we fix the policy the rest of the paper acts on. Table 1 traces its last two revisions on all four suites, with the per-chunk redraw and no tickets, under the evaluation protocol of §3.5.

The first revision replaced a decoder of 4 experts of 8 layers with 8 experts of 4 layers at an identical parameter count, and in the same run added a full-resolution token grid for the wrist camera, raised the loss weight on the gripper-phase transition, and stopped randomising one action dimension that had been randomised by mistake. Together these raised the four-suite average from 75.8% to 81.5%, with the largest moves on Goal (+6.5) and LIBERO-Long (+14.0). The second revision added a two-frame state history and raised the ResNet input resolution from 224 to 256 pixels, reaching 83.5%; its Goal gain (79.0% $\rightarrow$ 88.0%, 7 discordant states against 25, $p = 0.0021$) is the only individually significant suite-level move in the table.

Table 1: The base policy’s last two revisions, per-chunk redraw, no tickets, canonical states 0–19, 20 episodes per task. Both decoders have 32 DiT layers and the same parameter count (639.3M). The last row is the checkpoint used in every other experiment.

Decoder	Object	Spatial	Goal	Long	Avg
4 experts $\times$ 8 layers	87.0	87.5	72.5	56.0	75.8
8 experts $\times$ 4 layers, wrist grid, gripper-phase loss	86.5	90.5	79.0	70.0	81.5
+ 2-frame state history, 256 px ResNet input	86.5	88.5	88.0	71.0	83.5

Table 2: LIBERO-Long task 0, canonical states 0–19, frozen weights. Only the first and last rows are stable; the middle cycling row is two different random draws and is the warning of §4.4.

Noise at inference	Success
One random frozen vector ($m=1$)	3/20 (15%)
Four random vectors cycled ($m=4$)	7/20 and 14/20 (two draws)
Per-chunk redraw (unmodified policy)	11/20 (55%)
Four searched vectors cycled	16/20 (80%)

The first revision changed four things at once, so it does not isolate the number of experts. The per-task pattern – gains concentrated on opening a drawer, placing into a bowl and pushing a plate – points at the added wrist resolution at least as much as at routing, and a decomposition of the decoder’s error into a component shared by all experts and a component that varies across them found most of the residual error to be shared, which bounds what any change of expert topology can buy. We therefore claim only what Table 1 directly shows: the 8-expert configuration we use is the strongest of its lineage, by 7.7 points over the 4-expert configuration it replaced. That is the property the rest of the paper needs. The 71.0% LIBERO-Long baseline that the searched cycles improve on in §4.5 is the best base policy we had, not a weak one chosen to flatter an inference-time method. Appendix B lists the directions we measured and closed on this policy before turning to its initial noise.

4.3 One vector collapses long-horizon success

Table 2 gives the four conditions on LIBERO-Long task 0 (*put both the alphabet soup and the tomato sauce in the basket*) over the 20 reported states.

The 15% figure is not a single unlucky draw. An independent tiered search scored 64 distinct random vectors on one canonical layout, one episode each, and 8 of 64 passed – a 12.5% per-layout pass rate against the redraw’s 90% on that same layout. A second search measured 6%. These are estimates of the population mean over vectors, not of one vector, and they agree with 3/20.

This is the quantity that decides whether search is possible. A tier that requires 5 successes from 5 episodes is cleared roughly once per $1/p^5$ candidates: at $p = 0.15$ that is 13,169 candidates, and at $p = 0.70$ it is 6. At the long-horizon $m = 1$ rate, a search budget that finds a ticket on a short task finds nothing.

4.4 Cycling restores searchability but does not, by itself, help

The mechanism claim is a statement about populations of candidates, and we support it with the two measurements that are population quantities rather than single draws. Over pools of 64 and 128 candidates evaluated on the same layouts, the per-layout pass rate is **6% at $m=1$ and 61% at $m=4$** . The same budget that yields nothing at $m=1$ yields roughly 43 tier-1 survivors from 256 candidates at $m=4$.

If the deficit were about *which* vector is used, m should not matter: a random 4-tuple is no better chosen than a random 1-tuple. That it matters by an order of magnitude in pass rate says the binding constraint is temporal – consecutive chunks must not share noise – which is exactly the condition training never violates.

We deliberately do not quote a curve over m . An initial sweep at $m \in \{1, 2, 4, 8, 16, 32\}$ rose steeply to $m=4$ and was flat after, but each point was one draw, and a second random 4-tuple on the same task read

Table 3: LIBERO-Long, canonical states 0–19, frozen weights, paired by state. No single task reaches significance at $n=20$; the suite does.

Task	Per-chunk redraw	Searched 4-cycle	Δ	Discordant (win/loss)
T0	55%	80%	+25	6 / 1
T1	85%	65%	−20	2 / 6
T2	90%	100%	+10	2 / 0
T3	95%	100%	+5	1 / 0
T4	70%	70%	0	4 / 4
T5	80%	100%	+20	4 / 0
T6	55%	70%	+15	5 / 2
T7	85%	95%	+10	3 / 1
T8	40%	55%	+15	5 / 2
Suite	71.0%	79.0%	+8.0	32 / 16 , $p = 0.029$

Table 4: All four LIBERO suites, canonical states 0–19, frozen weights. A task without a ticket, or whose ticket loses to the redraw, runs the redraw.

Suite	Per-chunk redraw	With tickets	Ticket type
Object	86.5%	96.0%	single vector
Spatial	88.5%	97.5%	single vector
Goal	88.0%	94.5%	single vector
LIBERO-Long	71.0%	81.0%	searched 4-cycle
Average (4)	83.5%	92.3%	

35% where the sweep’s read 70%. The sweep located no knee and should not be cited as if it had; replicating it with several draws per m is listed in §5.

The control that matters. A random cycle is *not* better than the per-chunk redraw. Applying one unsearched 4-tuple to all ten LIBERO-Long tasks gives 63.5% against the redraw’s 71.0%, with the redraw winning 41 discordant layouts to 26 ($p = 0.086$). Everything the method delivers on this suite comes from the search; cycling supplies the regime in which a search can run.

4.5 Searched cycles on LIBERO-Long

Nine of LIBERO-Long’s ten tasks carry a searched 4-cycle, found with the procedure of §3.4 split across four machines and merged. Table 3 reports them on the 20 states no search saw.

Pooled over the nine tasks the cycle wins 32 of 48 discordant layouts ($p = 0.029$, two-sided sign test). T1 is the exception: its ticket costs 20 points. With T1 falling back to the redraw, the suite reaches 81.0% and the cycles win 30 discordant layouts to 10 ($p = 0.0022$); this is the configuration carried into Table 4.

A secondary observation: successful episodes get *faster* as m rises, 259 mean steps at $m=1$ against about 166 at $m=4$. The few episodes a single frozen vector does solve, it solves laboriously.

4.6 The three short suites

The short suites are context rather than the contribution – their episodes are short enough that a single frozen vector already works, which is the regime Patil et al. [12] reported. Table 4 gives our numbers.

Tickets lift every suite, by 6.5 to 10 points, and the four-suite average by 8.8 points, from 83.5% to 92.3%. Our search budget is an order of magnitude below that of Patil et al. [12], who report 100% on Object with 1081–1416 candidates per task; we expect the remaining short-suite gap to close with budget rather than with method, but have not shown it.

On three short tasks we also compared a random 4-cycle against the single ticket each already had: Goal T6 75% vs 70%, Goal T9 55% vs 70%, Object T4 80% vs 85%. None of the three differences is significant – they are one to three net episodes at $n=20$ – and they do not order monotonically with chunk count. We read them as consistent with the horizon story and as supplying no evidence for it. The one thing they do show is not statistical: on Goal T9 a random 4-cycle reaches the redraw’s rate *deterministically*, converting a sampled 70% into a guaranteed one. And on Goal T6 the searched single ticket beats the random 4-cycle, which is what should happen wherever the $m=1$ search succeeds. Cycling is worth reaching for where a single ticket underperforms, not as a general replacement.

4.7 Selection inflation and the two-batch control

A search rate is a maximum over candidates and is biased upward by construction. A ticket reported at 14/15 from a pool of 128 is entirely consistent with a true rate near 70%. Treating a search rate as a performance estimate is the single most common way to get a wrong number out of this method, and we did it before we stopped.

Three tickets the search ranked as winners did not beat the unmodified policy on states 0–19: LIBERO-Long T1 (65% against the redraw’s 85%), Goal T9 (55% against 70%), and Object T4 (a tie). Each had cleared every search tier. We found all three the same way, by pairing each ticketed task against the redraw on states the search tiers never used. That comparison is cheap – two batches – and was the most informative check in the project; a ticket that loses it falls back to the redraw. Our search also implements a stricter variant on states 35–49, run with a stride of 1 so that one batch covers distinct layouts, which banks a ticket only if it clears the unmodified policy there by a margin.

One diagnostic we tried and rejected: the gap between a ticket’s search rate and the baseline’s rate on the same layouts does not predict whether the ticket beats the unmodified policy on the reported states. It looked predictive at $n=2$ and was contradicted at $n=4$. We record it as a measured non-result so that others do not spend the compute.

4.8 What the tickets look like

We inspected the banked tickets against their candidate pools along four axes. The winners are indistinguishable from noise as fields; their norms are uncorrelated with their scores (so the effect is direction, not scale, and is not replaceable by a single scalar); the separation between top and bottom quartiles is flat across horizon positions rather than concentrated in the two executed rows; and the pairwise cosine similarity between tickets banked for different tasks is at the level expected of random vectors in 448 dimensions. Every task found its own direction, and we found no shared one. This is a negative result about interpretability, and it bounds what can be claimed: the search finds something real and task-specific, and we cannot say what it is.

5 Limitations

1. **One checkpoint, one architecture.** Nothing here separates a property of chunked flow matching from a property of this training run. The prediction the mechanism makes is specific and testable: any chunked flow-matching policy should show the collapse in proportion to its chunks per episode. We have not tested it on a second policy.
2. **The m curve is not located.** We report the $m=1$ versus $m=4$ contrast as a population quantity over candidate pools, which is sound, but the shape between and beyond is one draw per point and the second draw disagreed by 35 points. Why the knee appears to sit at 4 rather than 2 is unexplained; at $m=2$ chunks k and $k+2$ still share a vector, but that restates the data.
3. **The short-suite evidence for the horizon story is null.** Three short tasks land within noise. The horizon claim rests on LIBERO-Long, and within it chiefly on the pass-rate measurement.
4. **Post-search selection on the reported states.** For a minority of tasks the final ticket was chosen among the top candidates of its search pool by score on states 0–19, or replaced by the redraw where it

lost there (LIBERO-Long T1). Those per-task numbers are therefore optimistic relative to a strictly held-out protocol. LIBERO-Long with every searched ticket active, 79.0% at $p = 0.029$, involves no such step. For the short suites we expect a larger search to reach the same ceiling unaided, as Patil et al. [12] did with roughly ten times our budget, but we have not run it.

5. **Search cost.** Each candidate evaluation is a rollout. Our budgets (hundreds of candidates per task) are an order of magnitude below Patil et al.’s roughly 1300, so our short-suite numbers are not a like-for-like comparison with theirs and we do not present them as one.

6 Conclusion

The initial noise of a chunked flow-matching policy is a real, searchable, weight-free axis – but on long-horizon tasks only after the repetition constraint is removed. Freezing one vector fails there not because the vector is wrong but because a 76-chunk episode asks the model to condition on the same draw 76 times, which training never does. Cycling four fixed vectors removes that condition at no cost to determinism, raises the candidate pass rate from 6% to 61%, and makes the search feasible; the search then yields 71.0% $\rightarrow$ 79.0% on LIBERO-Long with frozen weights, and 81.0% once the one losing ticket falls back to the redraw. Cycling alone buys nothing, and we report the control that shows it. The broader lesson is procedural: in a method whose output is the argmax of a search, a search rate is not a performance estimate, and some tickets the search calls winners will be worse than doing nothing until they are checked against the unmodified policy on states the search did not use.

Reproducibility

Code, the ticket bundles, and the evaluation JSONs are at <https://github.com/wiltechs-hd/wilro-moe>. The checkpoint is public. Cycled rollouts are deterministic, so a correct reproduction reproduces the per-episode outcome vectors exactly, not merely the rates.

References

- [1] Johan Bjorek et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [2] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Khalil Meftah, Maxime Ellerbach, Jess Moss, and Thomas Wolf. LeRobot: State-of-the-art machine learning for real-world robotics in pytorch. <https://github.com/huggingface/lerobot>, 2024.
- [3] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023. URL <https://arxiv.org/abs/2303.04137>.
- [4] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.
- [5] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations*, 2019. arXiv:1803.03635.
- [6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020. arXiv:2006.11239.
- [7] Hugging Face TB Research. SmolVLM2-500M-Video-Instruct. <https://huggingface.co/HuggingFaceTB/SmolVLM2-500M-Video-Instruct>, 2025.

- [8] Zohar Karnin, Tomer Koren, and Oren Somekh. Almost optimal exploration in multi-armed bandits. In *International Conference on Machine Learning*, pages 1238–1246, 2013.
- [9] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. arXiv:2210.02747.
- [10] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [11] Jiafeng Mao, Xueting Wang, and Kiyoharu Aizawa. The lottery ticket hypothesis in denoising: Towards semantic-driven initialization. In *European Conference on Computer Vision*, pages 93–109, 2024. arXiv:2312.08872.
- [12] Omkar Patil, Ondrej Biza, Thomas Weng, Karl Schmeckpeper, Wil Thomason, Xiaohan Zhang, Kausik Sivakumar, Robin Walters, Nakul Gopalan, Sebastian Castro, Stephen Hart, and Eric Rosen. You’ve got a golden ticket: Improving generative robot policies with a single noise vector. *arXiv preprint arXiv:2603.15757*, 2026. URL <https://arxiv.org/abs/2603.15757>.
- [13] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *IEEE/CVF International Conference on Computer Vision*, 2023. arXiv:2212.09748.
- [14] Physical Intelligence et al. $\pi_{0.6}^*$: A VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025. URL <https://arxiv.org/abs/2511.14759>.
- [15] Allen Z. Ren et al. Diffusion policy policy optimization. *arXiv preprint arXiv:2409.00588*, 2024. URL <https://arxiv.org/abs/2409.00588>.
- [16] Mustafa Shukor et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. URL <https://arxiv.org/abs/2506.01844>.
- [17] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021. arXiv:2010.02502.
- [18] Andrew Wagenmaker, Mitsuhiro Nakamoto, Yunchu Zhang, Seohong Park, Waleed Yagoub, Anusha Nagabandi, Abhishek Gupta, and Sergey Levine. Steering your diffusion policy with latent space reinforcement learning. *arXiv preprint arXiv:2506.15799*, 2025. URL <https://arxiv.org/abs/2506.15799>.

A Architecture details

This appendix describes the checkpoint used throughout the paper, `ISdept/wilro-wilromoe-8x4-22k-obs2`. Shapes and parameter counts were measured by instantiating the released code under that checkpoint’s configuration. Notation: B batch, $H=64$ action horizon, $D=7$ action dimension, $d=960$ model width. Figure 1 gives the overall data flow and the structure of one decoder layer.

A.1 Encoder

The encoder is SmolVLM2-500M-Video-Instruct [7]: a SigLIP vision transformer, a pixel-shuffle connector, and a 32-layer language model of width 960 (15 query heads, 5 key–value heads, head dimension 64, FFN width 2560). It is frozen in its entirety and runs once per observation, not once per denoising step.

Each of the two LIBERO cameras (agent view and wrist) contributes its current frame, resized to 384×384 , which the vision tower and connector reduce to 36 tokens; the instruction is tokenized and embedded with the language model’s own frozen embedding table. The language model runs over the concatenation [image tokens | instruction tokens] – 72 image tokens plus the instruction – and two things are kept from that pass:

- the post-RoPE keys and values of *all 32* layers, detached from the graph, which the experts cross-attend into; and
- the mean-pooled final hidden state, which conditions the router.

No gradient reaches the encoder by either route.

A.2 Decoder input sequence

The decoder operates on one sequence per sample, under a causal mask:

Segment	Tokens	Source
sink	1	learned vector
robot state	2	current and previous 8-D state, each through a small MLP
agent-view camera	144	ResNet-18 feature map pooled to 12×12
wrist camera	256	ResNet-18 feature map at native 16×16
noisy actions x_t	64	Linear(7, 960) + learned position embedding
total	467	

The ResNet-18 is truncated after its third stage (3.03M parameters, trainable) and reads both cameras at 256×256 ; the same backbone serves both, and only the pooling differs, so the wrist view, which carries contact geometry, gets a denser grid at no parameter cost. Because there is no separate vision cross-attention, camera tokens are placed *before* the action tokens, where causal self-attention can reach them.

A.3 Experts

The head has 8 experts, each a stack of 4 DiT layers of width $d = 960$. Expert e ’s layer i cross-attends into VLM layer $4e + i$, so the eight experts partition the 32 VLM layers into disjoint, contiguous bands from shallow to deep. Setting the decoder width equal to the VLM’s hidden size is what lets a layer read the VLM’s cached keys and values without a projection.

Each DiT layer has three sublayers, each pre-normalized with RMSNorm and modulated by adaLN-Zero [13] from the timestep embedding:

1. causal self-attention over the 467-token sequence, with grouped-query attention (15 query heads, 5 key-value heads);
2. cross-attention whose queries are projected from the DiT stream and whose keys and values are the cached ones of the assigned VLM layer;
3. a SwiGLU feed-forward network, $960 \rightarrow 2560 \rightarrow 960$.

adaLN-Zero produces nine modulation vectors per layer (shift, scale and gate for each sublayer) and is initialized so that every residual branch starts at zero, which makes each expert the identity map at initialization.

One DiT layer	Parameters
self-attention (Q, K, V, O)	2.46M
cross-attention (Q, O)	1.84M
SwiGLU FFN	7.37M
adaLN-Zero modulation	8.30M
RMSNorms	<0.01M
total	19.98M

A.4 Router and mixing

Every expert runs on every sample; the mixture is dense, and its structure buys specialisation over VLM depth rather than compute. Each expert maps its action-token outputs through a shared final RMSNorm and Linear(960, 7) to a velocity v_e , and the policy’s velocity is

$$v = \sum_{e=1}^8 w_e v_e, \quad w = \text{softmax}(r),$$

where the router logits $r \in \mathbb{R}^8$ come from a two-layer MLP ($3840 \rightarrow 960 \rightarrow 8$, SiLU) over the concatenation of four 960-dimensional vectors: the state embedding, a projection of the VLM’s pooled final hidden state, the timestep embedding, and the mean of the noisy-action embeddings. During training only, $\mathcal{N}(0, 0.5^2)$ noise is added to the logits before the softmax to keep under-used experts receiving gradient, and a load-balancing term – the squared coefficient of variation of the batch-mean expert usage, weight 0.1 – is added to the loss. Inference uses the noise-free weights.

A.5 Flow matching and inference

The interpolant is $x_t = t x_1 + (1 - t) a$ with $t = 1$ at noise and $t = 0$ at data, the regression target is $x_1 - a$, and t is sampled uniformly during training. At inference the policy integrates the learned field with 10 explicit Euler steps on a uniform grid from $t = 1$ to $t = 0$, producing an $H \times D = 64 \times 7$ chunk, executes the first 2 actions, and replans. The cycling of §3.3 replaces only the starting point x_1 of this integration.

A.6 Training

States and actions are normalized to zero mean and unit variance with the dataset’s statistics; images enter unnormalized. The loss is the flow-matching mean-squared error, reweighted by a factor of 3 on the action steps within two steps of a gripper open/close transition, plus two auxiliary terms: the router load-balancing term above and a contrastive language term (weight 0.1). The contrastive term permutes the language segment of the cached keys and values across the batch, reruns the decoder with gradients disabled to obtain a velocity v^{wrong} under the wrong instruction, and applies a hinge $\max(0, 0.05 - \overline{(v - v^{\text{wrong}})^2})$ to samples whose instructions differ, so that the prediction must move when the instruction does. Regularization is dropout 0.1 in the decoder and, during training only, dropout with probability 0.15 of whole vision tokens, applied both to the image tokens entering the VLM and to the ResNet tokens. Each instruction is replaced by one of up to eight paraphrases.

The checkpoint was trained on `lerobot/libero`, all four suites, in two stages. Stage 1 trained the 8-expert decoder from scratch with single-frame state and the ResNet at 224×224 (64 agent-view and 196 wrist tokens); the middle row of Table 1 is this run’s final 30,000-step checkpoint. Stage 2 resumed from its 27,000-step checkpoint with the step counter reset, added the previous state frame, and raised the ResNet input to 256×256 with the token counts above. Stage 2 used Adam ($\beta = (0.9, 0.999)$, L_2 weight decay 10^{-6}), batch size 64, a peak learning rate of 10^{-4} with 1,500 linear warmup steps followed by a cosine decay over 25,000 steps, and no state-noise augmentation. The released checkpoint is its step 22,000, where the learning rate is 3.96×10^{-6} .

A.7 Parameter budget

Component	Trainable	Frozen
SigLIP vision tower		86.43M
connector		11.80M
language model (32 layers)		361.94M
experts (8×4 DiT layers)	639.35M	
router	4.62M	
ResNet-18 (to stage 3)	3.03M	
timestep embedder	1.85M	
state, action and output projections, sink, position embedding	0.09M	
total	648.93M	460.17M

The full policy has 1.11B parameters. adaLN-Zero is the largest single block in the decoder, larger than the feed-forward networks, because every layer carries a $\text{Linear}(960, 9 \times 960)$ that makes the whole stack time-conditioned.

B Axes measured and closed

Before turning to the initial noise, we spent most of this project trying to improve the policy itself. This appendix records the directions we measured and closed on this policy family, so that the noise intervention can be read against them: it was not the first thing tried, and the base policy of Table 1 is not one that obvious levers would have lifted. Unless stated otherwise, every comparison is paired by initial state on canonical states 0–19 with 20 episodes per task, against the checkpoint used in the rest of the paper.

How to read a null here. Two calibrations decide what counts as an effect. First, redrawing only the flow-noise stream of the *same* policy on the *same* states makes 20.5% of Goal episodes change outcome; a comparison whose discordance falls below that is indistinguishable from resampling. Second, two checkpoints of one run 3,000 steps apart, with no intervention between them, differed on Spatial by 85.0% against 91.5% ($p = 0.029$); their four-suite averages, 83.25% and 83.75%, pooled to 83.50%, the value of the checkpoint they were being compared against. A single significant per-suite comparison between two checkpoints is therefore not evidence of an intervention, and we did not treat it as one.

Table 5: Directions measured and closed. “Ours” is the checkpoint used throughout the paper (four-suite average 83.5%).

Axis	Intervention	Result
Input resolution	VLM input $384 \rightarrow 512$ px; ResNet $256 \rightarrow 320$ px with 400 wrist tokens	null: -2.25 and -0.75 points over 400 episodes each
Language grounding	contrastive language loss, hard negatives	already satisfied; prediction moves $1.4\text{--}2.0\times$ its own error
Expert topology	error decomposition over the 8 experts	at most 18.6% of the flow loss is reachable
Training budget	ten continuation runs from ours; two cosine anneals	no run improved; anneals null or negative
Loss reweighting	wider gripper window, down-weighted chunk tail	Goal $88.0 \rightarrow 86.0$, $p = 0.57$
Learning from own rollouts	AWR on 520 self-collected Goal episodes	Goal $88.0 \rightarrow 59.5$
Chunk-level selection	learned scorer picks 1 of 4 draws per chunk	identical to a random pick, $p = 1.0$
State-noise augmentation	training without it (earlier 4×8 decoder)	$+1.0$, $p = 0.85$

B.1 Input resolution

LIBERO renders its cameras at 256×256 , and the ResNet’s third stage has stride 16, so 256 tokens per camera at 256 px is a one-to-one read of the feature map. Three attempts to go beyond the native resolution of a component returned nothing. Raising the VLM’s input from 384 to 512 px, its native resolution, for about 40 hours of training moved Goal from 88.0% to 87.0% ($p = 0.86$, discordance 15.0%, below the resampling floor) and LIBERO-Long from 71.0% to 67.5%, -2.25 points pooled over 400 episodes; the share of gradient reaching the VLM cross-attention fell slightly with $1.78\times$ more vision tokens, so the decoder declined the extra capacity. Raising the ResNet input to 320 px with 400 wrist tokens moved Spatial and LIBERO-Long in opposite directions by equal amounts, 316 of 400 episodes against 319. And a one-to-one read of the agent-view camera, trained with single-frame state, scored 79.5% on Goal: 2.0 points below its parent and 8.5 below ours ($p = 0.0095$), despite carrying more agent-view detail than either. That last result also means the second revision in Table 1, which raised resolution and added the previous state frame together, cannot credit its gain to resolution.

B.2 Language grounding

The contrastive language term (Appendix A) printed 0.0000 on every run, which we first read as the term never engaging. It is the opposite: the hinge is satisfied with room to spare. With hard negatives switched on, permuting the instruction moved the predicted velocity by 139–203% of the model’s own flow-matching error, an order of magnitude above the margin. Consistent with this, the referential tasks that had looked like grounding failures – Spatial’s “next to the ramekin” against “on the ramekin”, and “next to the plate”, which had read exactly 70% on four different models – rose from 45–75% to 75–95% in the first decoder revision of Table 1, which did not touch the language pathway and ran with hard negatives off. In the rollout analysis below, no failure came from picking the wrong object. We stopped spending runs on language, instruction rewriting or contrastive pressure.

B.3 Expert topology

For a mixture, the flow-matching loss decomposes exactly into the part all experts get wrong together and the part on which they disagree, which a larger ensemble averages away. Measured on the 8-expert decoder, the shared part is 81.4% of the flow loss and the removable part 18.6%. More experts, deeper experts or a different partition of the VLM’s layers can only act on that 18.6%, and the 4-versus-8-expert question of §4.2 is a fraction of it. The remaining 81.4% is common to every expert and can only be moved by what they all read.

B.4 Training budget

Ten continuation runs from our checkpoint – changing resolution, loss weights, data and schedule – produced no improvement. Two cosine anneals measured the schedule’s tail directly: 21k $\rightarrow$ 30k steps on the 4-expert decoder was exactly null (13 discordant states each way, $p = 1.0$), and 25k $\rightarrow$ 30k on the 8-expert decoder lowered Goal from 81.5% to 79.0%, giving back precisely the tasks the preceding revision had gained.

B.5 Loss reweighting, and stalling as a symptom

Widening the window of up-weighted steps around gripper transitions from 2 to 4, and down-weighting each chunk beyond its first eight steps to 0.3, moved Goal from 88.0% to 86.0% ($p = 0.57$, discordance 14.0%). The run is informative for another reason. Failed episodes in this family spend long stretches motionless, which we had treated as the failure mechanism. This run halved the longest motionless stretch in failed episodes, from 22.6 s to 11.0 s, and the success rate did not move. Stalling is a symptom of an episode already lost, not its cause.

B.6 Learning from the policy’s own rollouts

Advantage-weighted regression on 520 Goal episodes collected by the policy itself, mixed with the original data, lowered Goal from 88.0% to 59.5% after 4,000 steps at a learning rate of 3×10^{-6} . The loss was confined to one class of task. Every contact-precision task collapsed: opening the drawer by its handle 90% $\rightarrow$ 45%, drawer and bowl 80% $\rightarrow$ 35%, pushing the plate 85% $\rightarrow$ 30%, cream cheese into the bowl 60% $\rightarrow$ 20%, wine bottle onto the rack 70% $\rightarrow$ 15%. Every task that places an object on an open surface held to within 15 points. Two explanations were ruled out: dropping failed episodes from the corpus changed one task by two episodes, and two other continuations from the same checkpoint cost only 1–2 points, so resuming from it is not itself fragile. We attribute the damage to the form of the objective: with non-negative weights, every sample, including a failure at low weight, pulls the policy towards its own action, so the objective can say “less of this” but never “not this”.

B.7 Chunk-level selection

A learned scorer that predicts steps-to-success from an observation and a candidate chunk (pairwise ranking accuracy 0.92) was used to pick one of four draws per chunk on the five hardest Goal tasks, 100 episodes per condition. Against one draw per chunk it read 77% $\rightarrow$ 82%, which would have been recorded as a positive result. The control decides it: picking one of the four draws *at random*, which is provably equivalent to a single draw, read 83%, and the scorer against the random pick was 82% against 83% (14 discordant states against 13, $p = 1.0$). Selecting among draws within a chunk buys nothing. This null is what pointed us to the episode level, where a different noise stream does flip outcomes, and from there to the tickets of the main text.

B.8 State-noise augmentation

Removing training-time noise on the robot state, measured on the earlier 4-expert decoder and forked at 14k steps rather than trained from scratch, changed Spatial by +1.0 point (13 discordant states against 15, $p = 0.85$). It narrowed the spread across tasks – the weakest task rose from 45% to 70% while fewer tasks reached 100% – without moving the mean.

B.9 Scaling the noise

On the 8-expert decoder before state history, scaling the initial noise by 0.9 and 0.8 instead of redrawing it moved Goal from 79.0% to 80.0% and 81.0% ($p = 0.83$ and 0.56). The direction was consistent and neither step was significant. We did not pursue it: a scalar applied to every draw changes how far the sample sits from the mean, while the tickets of the main text change which direction it points, and only the latter produced a measurable effect.

C Reproduction notes

Code, ticket bundles and evaluation outputs are at <https://github.com/wiltechs-hd/wilro-moe>. All commands below run from the repository’s `src/` directory.

C.1 What is pinned

Checkpoint	ISdept/wilro-wilromoe-8x4-22k-obs2, revision b61cf9a9
Training data	lerobot/libero, all four suites
Software	Python 3.10, PyTorch 2.7.1, LeRobot 0.4.0, robosuite 1.4.1, MuJoCo 3.1.6, LIBERO 0.1.0, Gymnasium 1.3.0, CUDA 12.6
Evaluation	20 episodes per task, canonical states 0–19, seed 10000, 10 Hz control, 2 executed actions per chunk, 10 Euler steps
Hardware	NVIDIA L4 for every reported number

Every evaluation writes its environment into the output JSON (library versions, GPU, and a short digest of both), together with the commit of the evaluation code. Two results should be compared only after checking those fields. Our reported runs used Transformers 5.17 for Object and Spatial with tickets, and 5.18 for every other result.

C.2 Commands

The unmodified policy, all four suites:

```
python eval_libero.py \
  --checkpoint ISdept/wilro-wilromoe-8x4-22k-obs2 \
  --suites libero_spatial libero_object libero_goal libero_10 \
  --episodes 20 --n_action_steps 2 --out baseline.json
```

With the released tickets for one suite, add `--noise_tickets <bundle>/golden_tickets.safetensors`. A single vector or an m -cycle can be tried directly with `--noise_ticket vec.npy` or `--noise_cycle cycle.npy`, where a cycle is an array of shape $(m, 64, 7)$. The random 4-cycle control of §4.4 is one such array drawn from a standard normal and applied to every task.

A search for one task – a single vector for a short suite, a 4-cycle for LIBERO-Long:

```
python search_golden_ticket.py \
  --checkpoint ISdept/wilro-wilromoe-8x4-22k-obs2 \
  --suites libero_goal --task_ids 6 \
  --tickets 256 --envs_per_tier 5 --tiers 3 \
  --certify_layouts 15 --out ./goal_tickets
```

```
python search_golden_ticket.py \
  --checkpoint ISdept/wilro-wilromoe-8x4-22k-obs2 \
  --suites libero_10 --task_ids 0 --cycle 4 \
  --tickets 128 --envs_per_tier 5 --tiers 3 \
  --certify_layouts 15 --out ./long_tickets
```

The number of candidates and the task list varied across our runs; the flags that define the protocol of §3.5 are the ones shown. Candidates are seeded per task, so two machines searching the same task draw the same pool and machines searching different tasks do not. A search writes its progress after every batch and resumes from it after an interruption, with the same result as an uninterrupted run; a GPU-free test of that property ships as `test_search_resume.py`. Bundles searched on separate machines are combined with `python ticket_bundle.py merge <out> <in> ...`, and `python ticket_bundle.py report <dir>` lists what a bundle holds.

C.3 Settings that change the result

Each of the following cost this project at least one result before it was caught. A reproduction that differs in any of them measures a different quantity.

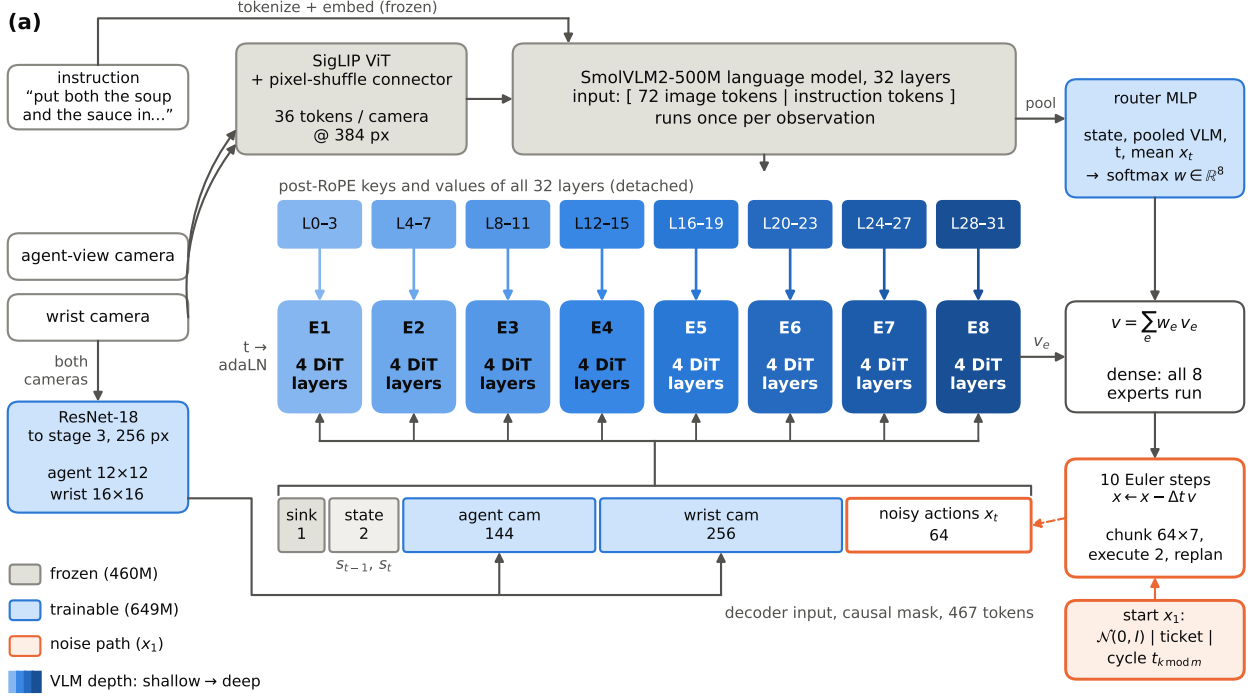
1. **Executed actions per chunk.** The checkpoint’s configuration ships `n_action_steps=64`, the full horizon, because that is what training uses. Evaluated that way, the policy replans a handful of times per episode and scores near zero. Every number in this paper uses `n_action_steps=2`; the evaluation script warns when it is not set.
2. **Control frequency.** The LIBERO demonstrations are recorded at 10 Hz, while LeRobot’s LIBERO environment defaults to 20 Hz. The evaluation and search scripts default to 10.
3. **Which layouts the environment serves.** LeRobot 0.4.0 restores a LIBERO initial state and *then* resets the simulator, and the reset re-samples object placements, discarding the canonical state. The layouts it serves come from LIBERO’s placement sampler, whose spread we measured at about ten times that of the 50 canonical states. Our scripts restore LIBERO’s own order (`libero_env_fixed.py`,

which can also check it); `--stock_init` reproduces the stock behaviour for comparison. Numbers measured under the two orders are not comparable.

4. **Which states are reported.** `--seed 10000` with `--init_state_offset 0` evaluates canonical states 0–19, the reported set. The search reads states 20–34 and certification 35–49. Reporting a ticket on any state its search read overstates it by construction (§4.7).
5. **Which noise the baseline drew.** With a ticket or a cycle the rollout does not depend on any seed. The per-chunk baseline draws its noise from `--policy_seed`, which defaults to `--seed`, so re-running it with the same seed reproduces every episode exactly, and changing only `--policy_seed` resamples it. The 20.5% discordance of Appendix B is that resampling.

C.4 Statistics

Every comparison in the paper pairs two runs by initial state. From the `episode_success` arrays in two output JSONs, count the states won only by the first run (b) and only by the second (c), pooled over the tasks being compared, and test b against $b + c$ with a two-sided binomial test at $p = 0.5$. Ties carry no information about the difference and are excluded.



(b) one DiT layer of expert e , layer i

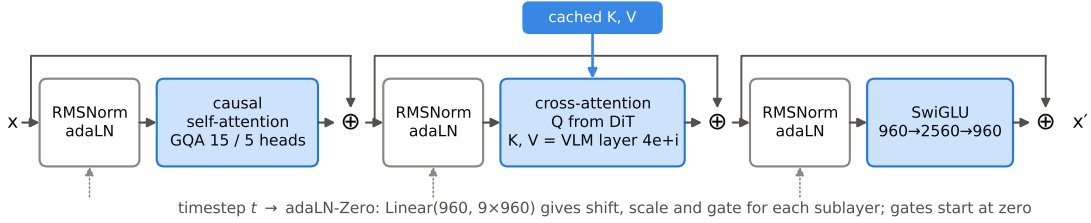


Figure 1: WILRO-MoE. **(a)** The frozen VLM (gray) encodes both camera images and the instruction once per observation and exposes the keys and values of all 32 of its layers. Each of the eight experts cross-attends into its own band of four consecutive VLM layers; color pairs each expert with its band, from shallow (light) to deep (dark). All experts share one decoder input sequence – a sink token, the current and previous robot state, ResNet tokens from both cameras, and the 64 noisy action tokens – and a router weights their velocity predictions. The orange path is the one this paper intervenes on: the initial noise x_1 from which the 10-step Euler integration starts, drawn fresh per chunk, fixed to a ticket, or cycled through m fixed vectors. **(b)** One DiT layer: pre-norm causal self-attention, cross-attention into the assigned VLM layer’s cached keys and values, and a SwiGLU feed-forward network, each modulated by adaLN-Zero from the timestep.